

MOBILE APPLICATION DEVELOPMENT WITH FLUTTER

WEEKEND PROJECT REPORT

Simple News Reader

Student Name: B MUNEZERO Ami Christian

Registration Number: 2401000232

Course: Mobile Application Development with Flutter

1. Introduction

The Simple News Reader is a Flutter-based mobile application developed as part of the Mobile Application Development with Flutter weekend project. The application provides a simple way for users to browse news articles, view information about an article, and open the complete article inside the application.

The application was developed using Flutter and Dart. Flutter widgets were used to create the user interface, while Riverpod was used to manage the application's news article state. The application contains three main screens: the Home Screen, Article Details Screen, and WebView Screen.

The project also demonstrates navigation, model objects, animations, form validation, error handling, JSON serialization, and state management. Two additional features were also implemented: deleting individual articles and clearing all articles.

2. Problem Statement

Many users need a simple way to browse news headlines and access complete articles without using a complicated interface. A news application should present information clearly and allow users to move between headlines and full articles easily.

The purpose of this project was to develop a simple news reader that demonstrates these functions while applying the Flutter and Dart concepts taught in the course. The application provides a list of articles, allows the user to select an article, displays its details, and provides a button for opening the full article.

3. Project Objectives

- To develop a functional application using Flutter and Dart.
- To display news articles using a scrollable list.
- To create a Dart model representing news articles.
- To implement Riverpod state management.
- To implement navigation between multiple screens and pass a model object between them.
- To implement Hero animation, AnimatedContainer and AnimatedOpacity.
- To implement form validation and an empty state.
- To implement a WebView for full articles.
- To provide user-facing error handling.
- To implement additional features beyond the basic requirements.

4. Tools and Technologies

4.1 Flutter

Flutter was the main framework for developing the application. It provided the widgets and tools required to create the user interface.

4.2 Dart

Dart was the programming language used for the model class, provider, screens, functions, and application logic.

4.3 Flutter Riverpod

Riverpod was used for state management. The project uses ProviderScope, Notifier, NotifierProvider, ref.watch(), and ref.read().

4.4 WebView

The `webview_flutter` package was used to open complete news articles inside the application.

4.5 Git and GitHub

Git was used to manage project versions and GitHub was used to store the project online.

4.6 Visual Studio Code

Visual Studio Code was the main development environment.

5. Application Structure

The application follows a simple folder structure:

```
lib/  
├── main.dart  
├── models/  
│   └── news_article.dart  
├── providers/  
│   └── news_provider.dart  
├── screens/  
│   ├── home_screen.dart  
│   ├── article_detail_screen.dart  
│   └── webview_screen.dart  
└── widgets/
```

- `main.dart` starts the application.
- The `models` folder contains the `NewsArticle` model.
- The `providers` folder contains the Riverpod state management code.
- The `screens` folder contains the three main screens.
- The `widgets` folder was prepared for reusable interface components.

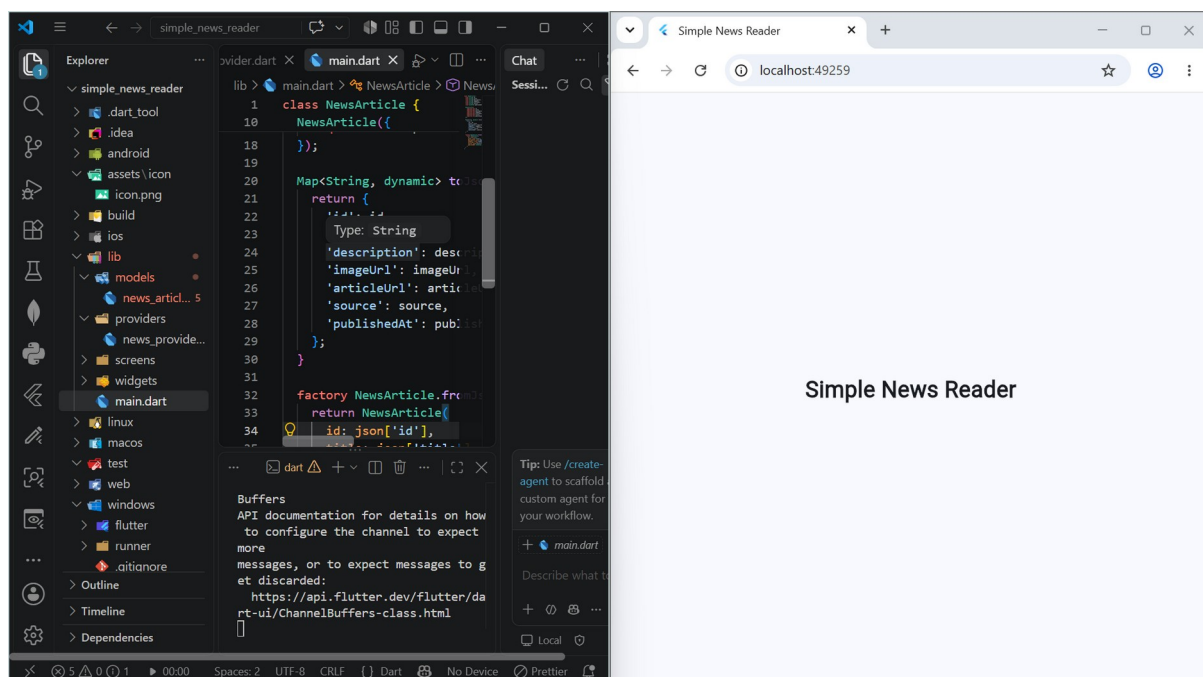


Figure 1: Project structure and `NewsArticle` model in VS Code, with the app running in Chrome

6. News Article Model

The NewsArticle class represents a news article. It contains these properties:

- ID
- Title
- Description
- Image URL
- Article URL
- Source
- Publication date

The model also contains toJson() and fromJson() methods. toJson() converts the article object into JSON-compatible data, while fromJson() creates a NewsArticle object from JSON data. This makes the application ready for future integration with an online news API or local storage.

7. Riverpod State Management

Riverpod was used to manage the application's list of news articles. The project contains a NewsNotifier that extends:

```
Notifier<List<NewsArticle>>
```

The provider is created using:

```
NotifierProvider<NewsNotifier, List<NewsArticle>>
```

The provider initially contains three sample news articles and has methods for adding an article, removing an article, and clearing all articles.

ref.watch() is used by the Home Screen to observe changes in the article list. ref.read() is used when calling addArticle(), removeArticle(), and clearArticles(). This allows the interface to update automatically when the article data changes.

8. Application Screens

8.1 Home Screen

The Home Screen is the main screen. It displays the available news articles using a ListView.builder. Each article shows an image, title, source, description, and a Delete button. The screen also has an Add button that opens the article form, and a menu option to clear all articles.

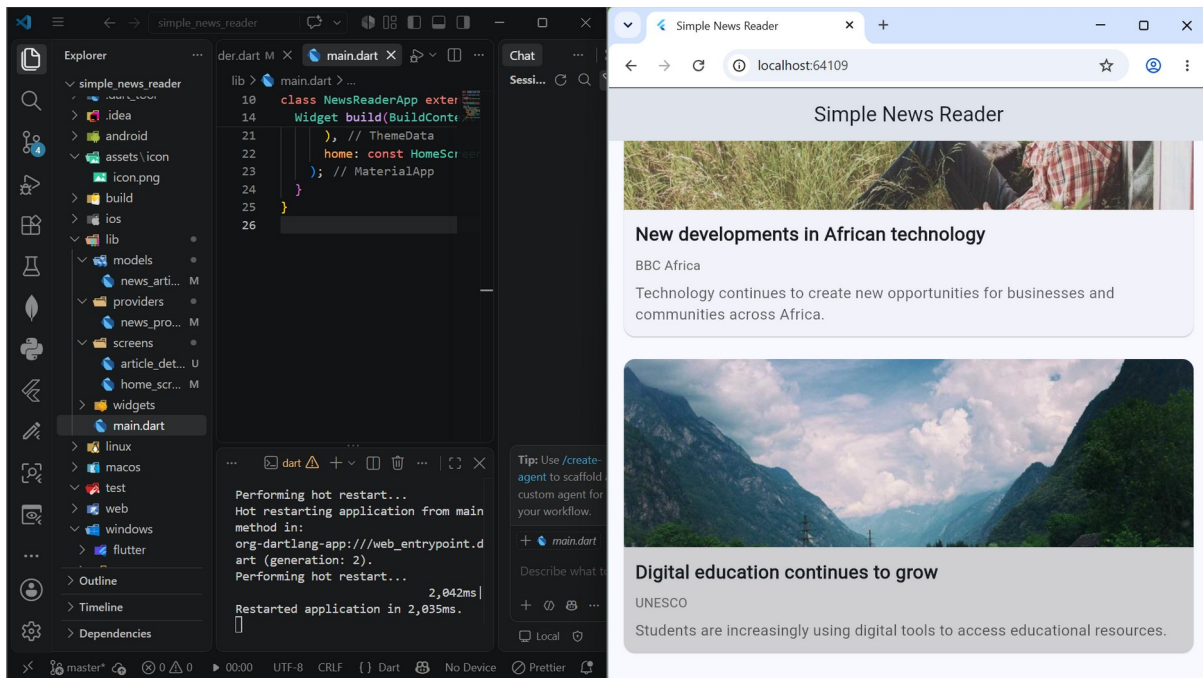


Figure 2: Home Screen showing the list of news articles

8.2 Article Details Screen

When the user selects an article, Navigator.push() opens the Article Details Screen and the selected NewsArticle object is passed to it. The screen displays the article image, title, source, description, a Read Full Article button, and a Back to News button. The image uses a Hero animation.

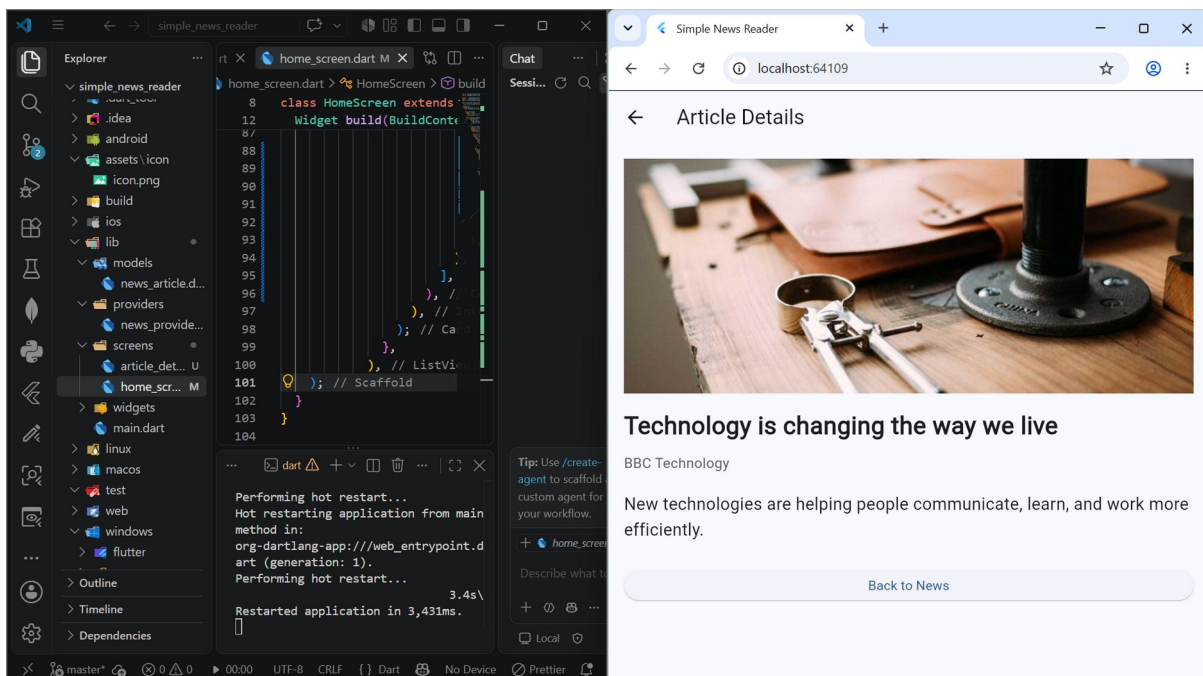


Figure 3: Article Details Screen

8.3 WebView Screen

The WebView Screen displays the complete article using the article URL. A WebViewController controls the WebView. A loading indicator is shown while the page

loads, and if an error occurs the application shows an error message with a Try Again button.

No separate WebView screenshot was captured, but the screen is implemented in the project.

9. Navigation and Data Passing

The application contains three screens. Navigation is implemented using `Navigator.push()` and `Navigator.pop()`. When an article is selected on the Home Screen, the Article Details Screen is pushed and the `NewsArticle` object is passed through the constructor. The Article Details Screen can then open the WebView Screen using another navigation operation. This demonstrates navigation between multiple screens and passing model objects between screens.

10. Animations

The project implements three animation techniques.

- `AnimatedContainer` is used around the article cards so visual changes occur smoothly.
- `AnimatedOpacity` is used for the article description, changing opacity depending on the state of the interface.
- `Hero` is used for the article image. The Home Screen and Article Details Screen use the same `Hero` tag, creating a smooth transition when an article is opened.

11. Form Validation

The application allows users to add a new news article. The form contains Title, Description, Image URL, Article URL, and Source fields. Before adding an article, the application checks whether all required fields have been completed. If a field is empty, the following message is displayed:

Please fill in all fields.

If all fields are completed, a new `NewsArticle` object is created and added through the Riverpod provider, and a success `SnackBar` is displayed.

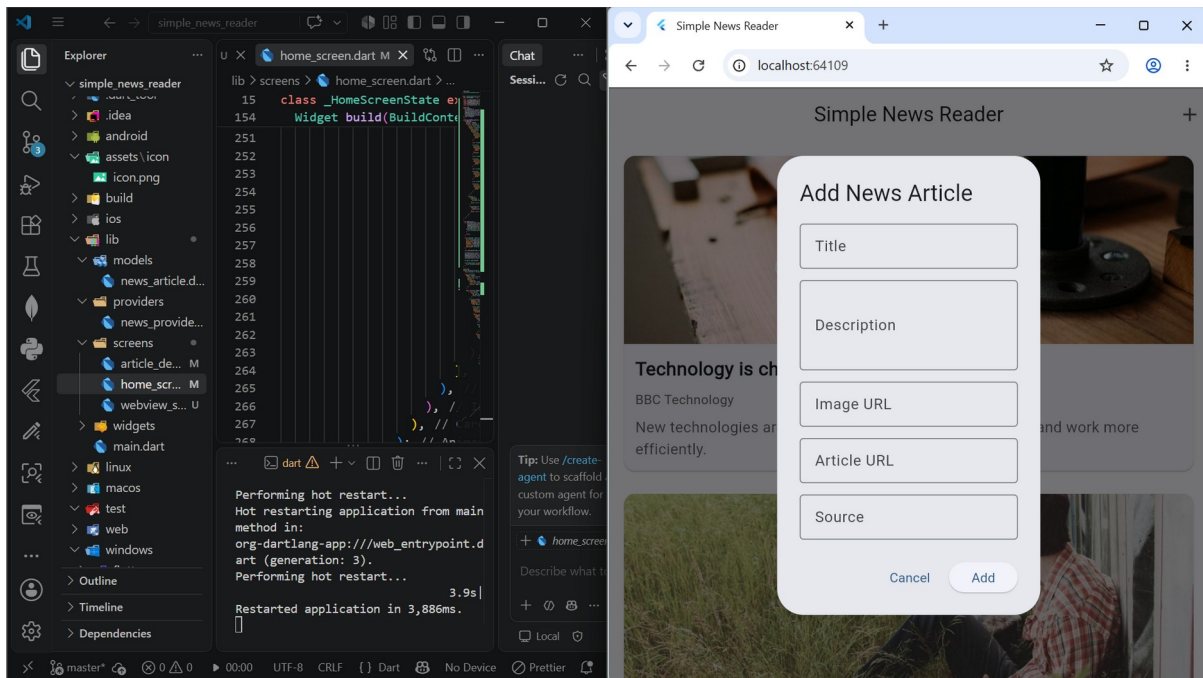


Figure 4: Add News Article form

12. Empty State

When all articles have been deleted or cleared, the user sees the message "No news articles available." This prevents a blank screen and informs the user about the current state of the application.

13. Error Handling

- If an image fails to load, the application displays "Image could not be loaded".
- If the WebView fails, the application displays "Unable to load this article." together with a Snackbar.
- The user can select Try Again to reload the article.

14. Additional Features

The assignment required at least two additional features. Two were implemented.

14.1 Delete Article

A Delete button was added to every article. Selecting it removes the article from the Riverpod state, and a Snackbar informs the user.

14.2 Clear All Articles

A Clear All Articles option was added to the menu. A confirmation dialog is shown first; after confirmation all articles are removed and the empty-state message is displayed.

15. Milestone Implementation

M1 — Project Setup and Custom Launcher Icon

The Flutter project was created, the folder structure was prepared, dependencies were added, and the custom launcher icon was configured using the Flutter Launcher Icons package. The project runs in Chrome (Figure 1).

M2 — Model Class

The NewsArticle model was created with the article properties and JSON serialization methods (Figure 1).

M3 — Riverpod Provider

The NewsNotifier and NotifierProvider were created to manage the article list, with methods for adding, removing, and clearing articles.

M4 — Home Screen

The Home Screen was created using Scaffold, AppBar, Column, Container, Row, and ListView, with an empty state (Figure 2).

M5 — Navigation and Data Passing

The Article Details Screen was created. Navigator.push() opens it, Navigator.pop() returns to the previous screen, and the NewsArticle object is passed between the screens (Figure 3).

M6 — Animations

AnimatedContainer, AnimatedOpacity, and Hero were added. The Hero animation connects the article image between the Home Screen and the Article Details Screen (Figures 2 and 3).

M7 — WebView, Validation and Error Handling

The WebView screen was implemented using webview_flutter. Form validation prevents incomplete articles from being added (Figure 4), and error handling was added for failed image and article loading. No separate WebView screenshot was captured, but the functionality is present in the project.

M8 — Final Polish and Testing

The project was reviewed and the two additional features, Delete Article and Clear All Articles, were added. The project was checked using:

```
flutter analyze
```

The analysis completed without Dart or Flutter errors. Android APK generation was not completed because the Android SDK was not available on the development computer.

16. Testing

The application was tested through different user actions.

Test	Expected Result	Result
Open application	Home Screen appears	Passed
View articles	Articles appear in list	Passed

Test	Expected Result	Result
Select article	Details Screen opens	Passed
Return to Home	Home Screen appears	Passed
Add valid article	Article is added	Passed
Submit empty form	Error message appears	Passed
Delete article	Article is removed	Passed
Clear all	All articles are removed	Passed
Empty state	Empty message appears	Passed
Image failure	Error message appears	Implemented
WebView loading	Article page opens	Implemented
WebView failure	Error message appears	Implemented
Flutter analysis	No analysis errors	Passed
Android APK	APK generated	Not completed

17. Challenges Encountered

17.1 Pubspec Configuration

A duplicate dependencies section in pubspec.yaml caused a YAML error. It was fixed by removing the duplicate section and keeping one correct dependencies section. This helped me understand the importance of correct YAML structure and indentation.

17.2 Riverpod

It took time to understand the difference between `ref.watch()` and `ref.read()`. `ref.watch()` listens for changes to the article list, while `ref.read()` calls methods on the notifier.

17.3 WebView and Platform Support

The WebView is implemented, but the development computer did not have a valid Android SDK installation. As a result, Android release testing and APK generation could not be completed.

18. Limitations

18.1 No Permanent Storage

Articles added by the user exist only during the current session and are lost when the application restarts.

18.2 Sample News Data

The application uses sample articles instead of a live news API.

18.3 Android Release

The Android release APK was not generated because the Android SDK was unavailable, so final Android-device testing was not completed. The application was tested in Chrome.

19. Future Improvements

- Connect to a live news API for current headlines.
- Add SharedPreferences or a database for permanent storage.
- Add a search feature to find articles by title.
- Add categories such as technology, sports, business, and world news.
- Test on Android devices and package a release APK.

20. Conclusion

The Simple News Reader project applies important Flutter and Dart concepts from the Mobile Application Development course. The application contains a news article list, article details, navigation, WebView functionality, Riverpod state management, form validation, empty-state handling, error handling, and animations. It also includes two additional features: deleting individual articles and clearing all articles with confirmation.

The project was successfully analysed using Flutter's analysis tools. However, Android APK generation and Android-device testing were not completed because the Android SDK was not available. The project provides a foundation that could be extended with permanent storage, live news data, search, categories, and Android release support.

21. References

1. Flutter Documentation — Flutter framework and widgets.
2. Dart Documentation — Dart programming language.
3. Riverpod Documentation — State management.
4. `webview_flutter` Package Documentation — WebView implementation.
5. Flutter Launcher Icons Documentation — Custom application launcher icons.